

**LAB 3****INSTRUCTIONS:**

- Given the backup of a MongoDB database *metagenomics.json*, you are required to import this database to work on the Lab. Use *mongoimport* to import the lab to your MongoDB Atlas / local MongoDB.
- The API endpoints should fulfil the tasks listed below.
- Please answer ALL FIFTEEN (15) questions.

**TASKS:****Question 1: List Samples with Filtering and Pagination****Endpoint:** `GET /samples`

Implement an endpoint that returns a paginated list of all samples. The caller must be able to filter by environment type. Each result should include `sample_id`, location (city & country), `collection_date`, `environment_type`, and the total sequence count for that sample.

**Parameters:**

| Parameter                     | Type          | Required | Default | Description                             |
|-------------------------------|---------------|----------|---------|-----------------------------------------|
| <code>page</code>             | <i>int</i>    | No       | 1       | Page number (1-indexed)                 |
| <code>limit</code>            | <i>int</i>    | No       | 20      | Number of results per page (max 100)    |
| <code>environment_type</code> | <i>string</i> | No       | —       | Filter by Soil, Water, Air, or Sediment |

**Expected Response (example):**

```
{ "total": 300, "page": 1, "limit": 20, "data":
[ { "sample_id": "SMP0001", "location": { "city": "Stockholm",
"country": "Sweden" }, "collection_date": "2018-04-13",
"environment_type": "Air", "sequence_count": 5 }, ... ] }
```

**Hints:**

- Use MongoDB `$.size` on the sequences array to compute `sequence_count`.
- Use `.skip()` and `.limit()` for pagination.

## Question 2: Get Full Sample Detail

**Endpoint:** `GET /samples/{sample_id}`

Retrieve the complete document for a given sample including all embedded sequences and their full taxonomy subdocuments. Return a 404 with a clear error message if the `sample_id` does not exist.

**Parameters:**

| Parameter              | Type          | Required   | Default | Description  |
|------------------------|---------------|------------|---------|--------------|
| <code>sample_id</code> | <i>string</i> | <b>Yes</b> | —       | e.g. SMP0001 |

**Expected Response (example):**

```
{ "sample_id": "SMP0001", "location": { "city": "...",
"country": "..." }, "collection_date": "...",
"environment_type": "...", "sequences": [ { "sequence_id":
"SEQ000001", "sequence": "ATCG...", "taxonomy":
{ "taxonomy_id": "TAX005", "kingdom": "Bacteria", ... } } ] }
```

**Hints:**

- Use `find_one({'sample_id': sample_id})`.
- Raise `HTTPException(status_code=404)` if result is `None`.

## Question 3: Get Sequences for a Sample

**Endpoint:** `GET /samples/{sample_id}/sequences`

Return only the sequences array from a given sample document. This endpoint is useful for clients that need sequence data without reloading the full sample. Return a 404 if the sample does not exist.

**Parameters:**

| Parameter              | Type          | Required   | Default | Description  |
|------------------------|---------------|------------|---------|--------------|
| <code>sample_id</code> | <i>string</i> | <b>Yes</b> | —       | e.g. SMP0042 |

**Expected Response (example):**

```
{ "sample_id": "SMP0042", "sequence_count": 4, "sequences":
[ { "sequence_id": "...", "sequence": "...", "taxonomy":
{ ... } } ] }
```

**Hints:**

- Include `sequence_count` in the response using ``len(doc['sequences'])``.

**Question 4: Search Samples by Location****Endpoint:** `GET /samples/search`

Allow callers to search for samples by country and/or city. Both parameters are optional but at least one must be provided. The search should be case-insensitive. Return a list of matching samples with their location and environment type.

**Parameters:**

| Parameter      | Type          | Required | Default | Description                              |
|----------------|---------------|----------|---------|------------------------------------------|
| <b>country</b> | <i>string</i> | No       | —       | e.g. Malaysia, sweden (case-insensitive) |
| <b>city</b>    | <i>string</i> | No       | —       | e.g. Manaus (case-insensitive)           |

**Expected Response (example):**

```
{ "count": 7, "data": [ { "sample_id": "SMP0033", "location":  
  { "city": "Kuala Lumpur", "country": "Malaysia" },  
  "environment_type": "Soil", "collection_date":  
  "... " }, ... ] }
```

**Hints:**

- Build the filter dict dynamically: only add keys if the parameter is provided.
- Use regex for case-insensitive matching: ``{ "location.country": { "$regex": country, "$options": "i" } }``.
- Return 400 if neither parameter is supplied.

**Question 5: Get a Single Sequence by ID****Endpoint:** `GET /sequences/{sequence_id}`

Retrieve a single sequence document by its `sequence_id`, regardless of which sample it belongs to. The response should include the parent `sample_id`, the full sequence string, and the taxonomy subdocument.

**Parameters:**

| Parameter          | Type          | Required   | Default | Description    |
|--------------------|---------------|------------|---------|----------------|
| <b>sequence_id</b> | <i>string</i> | <b>Yes</b> | —       | e.g. SEQ000042 |

**Expected Response (example):**

```
{ "sequence_id": "SEQ000042", "sample_id": "SMP0010",
  "sequence": "ATCG...", "taxonomy": { "taxonomy_id": "TAX003",
    "kingdom": "Bacteria", ... } }
```

**Hints:**

- Use ``$elemMatch`` or ``$unwind`` in an aggregation pipeline to locate the target sequence across all samples.
- Projection after ``$unwind``: ``{ "sequences": { "$elemMatch": { "sequence_id": seq_id } } }``.

**Question 6: List and Filter Taxonomy Records****Endpoint:**

- **GET** `/taxonomy`
- **GET** `/taxonomy/{taxonomy_id}`

Implement two sub-tasks: (a) list all taxonomy records, optionally filtered by kingdom (Bacteria or Archaea); (b) retrieve a single taxonomy record by `taxonomy_id`. Use the taxonomy data embedded within sample documents — do NOT create a separate collection.

**Parameters:**

| Parameter          | Type          | Required   | Default | Description                                                  |
|--------------------|---------------|------------|---------|--------------------------------------------------------------|
| <b>kingdom</b>     | <i>string</i> | No         | —       | Filter by Bacteria or Archaea (query param on list endpoint) |
| <b>taxonomy_id</b> | <i>string</i> | <b>Yes</b> | —       | e.g. TAX005 (path param on detail endpoint)                  |

**Expected Response (example):**

```
List: { "count": 18, "data": [ { "taxonomy_id": "TAX001",
"kingdom": "Bacteria", "phylum": "Proteobacteria", ... } ] }
Detail: { "taxonomy_id": "TAX005", "kingdom": "Bacteria",
"phylum": "Actinobacteria", "class": "Actinomycetia", ... }
```

**Hints:**

- Use an aggregation pipeline with `\$unwind` on sequences, then `\$group` on taxonomy\_id to get distinct records.
- Filter by kingdom using `\$match` after `\$unwind`.
- Return 404 if the taxonomy\_id is not found in any document.

**Question 7: Species Diversity per Environment Type****Endpoint:** `GET /analytics/diversity`

For each environment type, return the count of distinct species found across all sequences. If environment\_type is provided, return diversity stats for that type only. This helps researchers compare biodiversity across habitats.

**Parameters:**

| Parameter               | Type          | Required | Default | Description                                |
|-------------------------|---------------|----------|---------|--------------------------------------------|
| <b>environment_type</b> | <i>string</i> | No       | —       | If omitted, return results for all 4 types |

**Expected Response (example):**

```
{ "data": [ { "environment_type": "Soil",
"unique_species_count": 9 }, { "environment_type": "Water",
"unique_species_count": 8 }, ... ] }
```

**Hints:**

- Pipeline: `$match` (optional) → `$unwind` sequences → `$group` by {environment\_type, species} → `$group` by environment\_type with `$sum`.
- Use `$addToSet` in the first `$group` to collect distinct species, then `$size` in the second `$group`.

### Question 8: Sample Collection Trend Over Time

**Endpoint:** `GET /analytics/trend`

Given a date range, return the number of samples collected per month. This produces a time-series that allows researchers to see when sampling activity was highest. Both `start_date` and `end_date` are required.

**Parameters:**

| Parameter               | Type                          | Required   | Default | Description                |
|-------------------------|-------------------------------|------------|---------|----------------------------|
| <code>start_date</code> | <i>string</i><br>(YYYY-MM-DD) | <b>Yes</b> | —       | Range start date inclusive |
| <code>end_date</code>   | <i>string</i><br>(YYYY-MM-DD) | <b>Yes</b> | —       | Range end date inclusive   |

**Expected Response (example):**

```
{ "start_date": "2020-01-01", "end_date": "2021-12-31",  
  "data": [ { "year": 2020, "month": 3, "count": 6 }, { "year":  
2020, "month": 5, "count": 4 }, ... ] }
```

**Hints:**

- `collection_date` is stored as a string (YYYY-MM-DD); use ``$dateFromString`` inside ``$addFields`` to convert it before ``$match``.
- Use ``$dateToString`` with format ``"%Y-%m"`` or extract ``$year`` / ``$month`` in ``$group``.
- Return Error 400 if `start_date > end_date`. (Make sure provide proper description of the error)

### Question 9: Top N Most Frequently Occurring Species

**Endpoint:** `GET /analytics/top-species`

Return the top N species ranked by total number of sequence occurrences across the entire database. Include the full taxonomy detail (genus, phylum, kingdom) alongside the count. Default N is 10.

**Parameters:**

| Parameter | Type       | Required | Default | Description                              |
|-----------|------------|----------|---------|------------------------------------------|
| <b>n</b>  | <i>int</i> | No       | 10      | Number of top species to return (max 20) |

**Expected Response (example):**

```
{ "n": 5, "data": [ { "rank": 1, "species": "Pseudomonas aeruginosa", "genus": "Pseudomonas", "phylum": "Proteobacteria", "kingdom": "Bacteria", "count": 94 }, ... ] }
```

**Hints:**

- Pipeline: \$unwind sequences → \$group by species (collect taxonomy fields) → \$sort by count desc → \$limit n.
- Use `\$first` inside `\$group` to carry along genus, phylum, kingdom from the taxonomy subdocument.

**Question 10: Find Samples Containing a Specific Species****Endpoint:** `GET /analytics/species-search`

Given a species name, return all samples that contain at least one sequence classified under that species. For each matching sample, include the sample metadata and the list of matching sequence\_ids only (not all sequences).

**Parameters:**

| Parameter      | Type          | Required   | Default | Description                                      |
|----------------|---------------|------------|---------|--------------------------------------------------|
| <b>species</b> | <i>string</i> | <b>Yes</b> | —       | Exact or partial species name (case-insensitive) |

**Expected Response (example):**

```
{ "species": "Escherichia coli", "sample_count": 12, "data": [ { "sample_id": "SMP0005", "location": { "city": "...", "country": "..." }, "environment_type": "Water", "matching_sequence_ids": ["SEQ000031", "SEQ000032"] }, ... ] }
```

**Hints:**

- Use ``$filter`` on the sequences array to extract only sequences matching the species before projecting.
- Case-insensitive regex: ``{"sequences.taxonomy.species": { "$regex": species, "$options": "i" } }``.
- Avoid returning the full sequence strings — project only `sequence_id` from matching sequences.

**Question 11: Microbial Kingdom Breakdown by Country****Endpoint:** `GET /analytics/kingdom-by-country`

For each country (or a specific country if provided), return a breakdown of how many sequences belong to Bacteria vs Archaea. This allows geographic comparison of prokaryotic composition.

**Parameters:**

| Parameter      | Type          | Required | Default | Description                               |
|----------------|---------------|----------|---------|-------------------------------------------|
| <b>country</b> | <i>string</i> | No       | —       | If omitted, return data for all countries |

**Expected Response (example):**

```
{ "data": [ { "country": "Brazil", "Bacteria": 142, "Archaea": 18 }, { "country": "Sweden", "Bacteria": 89, "Archaea": 7 }, ... ] }
```

**Hints:**

- Pipeline: `$match` (optional) → `$unwind` sequences → `$group by {country, kingdom}` → `$group by country` pivoting kingdom counts.
- Use ``$cond`` with ``$eq`` on kingdom to split counts in a single ``$group`` stage: ``{ $sum: { $cond: [{ $eq: ['$kingdom', 'Bacteria'] }, 1, 0] } }``.

**Question 12: Samples Exceeding Species Diversity Threshold****Endpoint:** `GET /analytics/diverse-samples`

Return all samples where the number of distinct species within that single sample is greater than or equal to the given threshold `k`. This identifies highly diverse samples. Include the sample metadata and the actual distinct species count.



**Parameters:**

| Parameter          | Type       | Required   | Default | Description                                    |
|--------------------|------------|------------|---------|------------------------------------------------|
| <b>min_species</b> | <i>int</i> | <b>Yes</b> | —       | Minimum number of distinct species in a sample |

**Expected Response (example):**

```
{ "min_species": 5, "count": 23, "data": [ { "sample_id":
"SMP0017", "location": { "city": "...", "country": "..." },
"environment_type": "Soil", "distinct_species_count":
6 }, ... ] }
```

**Hints:**

- Pipeline: \$unwind → \$group by {sample\_id, species} (deduplicate) → \$group by sample\_id with \$sum → \$match count >= min\_species.
- Use `\$addToSet` on species per sample in the first group, then `\$size` in a `\$project` stage before filtering.
- Return ERROR 422 if min\_species < 1 (make sure provide proper description).

**Question 13: Phylum Distribution for a Specific Location**

**Endpoint:** **GET** /analytics/phylum-by-location

Given a city and country, aggregate all sequences from samples collected at that location and return a breakdown by phylum — how many sequences fall under each phylum. Useful for location-specific microbiome profiling.

**Parameters:**

| Parameter      | Type          | Required   | Default | Description                    |
|----------------|---------------|------------|---------|--------------------------------|
| <b>city</b>    | <i>string</i> | <b>Yes</b> | —       | e.g. Manaus (case-insensitive) |
| <b>country</b> | <i>string</i> | <b>Yes</b> | —       | e.g. Brazil (case-insensitive) |

**Expected Response (example):**

```
{ "location": { "city": "Manaus", "country": "Brazil" },
"total_sequences": 47, "phylum_breakdown": [ { "phylum":
"Proteobacteria", "count": 18 }, { "phylum": "Firmicutes",
"count": 12 }, ... ] }
```

**Hints:**

- Match on location using case-insensitive regex on both `location.city` and `location.country`.
- Pipeline: `$match` → `$unwind` → `$group` by phylum → `$sort` by count desc.
- Return 404 with a meaningful message if the location yields zero samples.

**Question 14: Sequence Length Statistics per Environment Type****Endpoint:** `GET /analytics/sequence-stats`

Compute the average, minimum, and maximum DNA sequence character length for sequences grouped by environment type. If `environment_type` is specified, return stats for that type only. This helps assess sequencing depth consistency across environments.

**Parameters:**

| Parameter                     | Type          | Required | Default | Description                                        |
|-------------------------------|---------------|----------|---------|----------------------------------------------------|
| <code>environment_type</code> | <i>string</i> | No       | —       | If omitted, return stats for all environment types |

**Expected Response (example):**

```
{ "data": [ { "environment_type": "Soil", "avg_length": 351.4,
"min_length": 201, "max_length": 498, "total_sequences":
382 }, ... ] }
```

**Hints:**

- Use `$strLenCP` inside `$addFields` after `$unwind` to compute sequence length as a numeric field.
- Then `$group` by `environment_type` using `$avg`, `$min`, `$max`, `$sum`.
- Round `avg_length` to 1 decimal using `$round: ['$avg_length', 1]` in a `$project` stage.

**Question 15: Multi-filter Analytical Summary per Sample****Endpoint:** `GET /samples/summary`

Return a filtered, summarised list of samples combining multiple optional query filters: `country`, `environment_type`, `start_date`, `end_date`, and `min_sequences`. Each result should include sample metadata plus `sequence_count`, `distinct_species_count`, and `dominant_species` (the species appearing most in that sample). This is the most complex endpoint where it combines all previously learned concepts.

**Parameters:**

| Parameter               | Type                        | Required | Default | Description                                           |
|-------------------------|-----------------------------|----------|---------|-------------------------------------------------------|
| <b>country</b>          | <i>string</i>               | No       | —       | Filter by country                                     |
| <b>environment_type</b> | <i>string</i>               | No       | —       | Filter by environment type                            |
| <b>start_date</b>       | <i>string</i><br>YYYY-MM-DD | No       | —       | Collection date range start                           |
| <b>end_date</b>         | <i>string</i><br>YYYY-MM-DD | No       | —       | Collection date range end                             |
| <b>min_sequences</b>    | <i>int</i>                  | No       | 1       | Only return samples with at least this many sequences |

**Expected Response (example):**

```
{ "filters_applied": { "country": "Brazil",
"environment_type": "Soil", "start_date": "2020-01-01",
"end_date": "2023-12-31", "min_sequences": 4 }, "count": 8,
"data": [ { "sample_id": "SMP0012", "location": { "city":
"Manaus", "country": "Brazil" }, "collection_date": "2021-06-
14", "environment_type": "Soil", "sequence_count": 6,
"distinct_species_count": 4, "dominant_species": "Streptomyces
griseus" }, ... ] }
```

**Hints:**

- Build the `\$match` stage dynamically — only include keys where the query param was provided.
- Use `\$dateFromString` for date comparison when filtering on collection\_date.
- Compute dominant\_species: \$unwind → \$group by {sample\_id, species} with \$sum → \$sort → \$group by sample\_id with \$first.

This endpoint benefits from being decomposed into helper functions — keep the route handler clean.